**Question 1: [2 marks] (0.5 mark each)**

- Missing value replacement based on most frequent value.

No. There are no missing values in the categorical and binary attributes, so no need to use missing values replacement based on most frequent value.

- Missing value replacement based on median value.

Yes. There are a reasonable number of missing values in numerical attributes 2 and 5 (15 and 23 out of 10 000) + probably some outliers (e.g., large max and min w.r.t. mean for attribute 2), therefore it seems reasonable to use missing value replacement based on median value for these 2 attributes.

- Removing an attribute.

Yes. There is a large majority of missing values in numerical attribute 4 (9995 out of 10 000), therefore it makes sense to remove this attribute completely. Doing missing value replacement might create misleading information.

- Outlier removal.

Yes. Looking at attribute 2 statistics, the extremums are very large compared to the mean and std of the normal distribution of the values. This is a clue that there is likely to be some outliers, and therefore outlier removal might be useful.

**Question 2 [2 marks]**

**(a) Model's performance in terms of bias and variance [1 mark] (0.5 for variance, 0.5 for bias)**

The model displays high variance as there is a clear gap between the training loss/accuracy and the validation loss/accuracy after training. This is also known as overfitting, where the model is becoming too good at predicting the training set, but is not able to generalise as well on the validation data.

It is harder to describe the bias at this point of training, as the training loss has not totally stabilised yet. However, it seems like the training loss is still relatively high (and the training accuracy low), being an int of a high bias. This means that the model does not fit well to the training data, maybe because it is not complex enough. However, this also depends on the complexity of the problem and what loss/accuracy can reasonably be reached (avoidable bias, i.e., 70% accuracy might be good for this task).

**(b) 2 possible changes among (non-exhaustive list) [2 marks]**

- Applying regularisation techniques can help reduce the overfitting/high variance and might lead to a better validation accuracy. These techniques help limit the complexity of the model, preventing to fit the training data too closely.
- Data augmentation might help with overfitting as well as high bias. Given the augmentation is relevant and not redundant, it should help the model fit the problem better and lower the risk of overfitting as more various data is seen during training.
- Changing the model size can help if we consider the model having a high bias. It will give more "freedom" to the model to fit the data better. However, it might lead to a higher variance in return.
- Batch normalisation can have a regularising effect as it introduces some noise in the activations, similarly to techniques like dropout.

**Question 3 [3 marks]**

(a) **LeakyReLU (1 mark):**

A potential problem with ReLU is the dying ReLU problem, where neurons falling in the negative activation zone stop participating in the predictions and the training. This is because ReLU output is 0 for negative inputs. (0.5 mark)

LeakyReLU mitigates this issue by adding a small slope to the negative part of the ReLU function, making the activation small but not equal to 0. This allows neurons falling in the negative activation zone to not die totally and still receive a little bit of signal to update their weights. (0.5 mark)

(b) **Output activation function (2 marks):**

Since each image can contain multiple anomalies simultaneously, this is a multi-label classification problem (as opposed to multi-class, where only one label is active per input). In this case, the appropriate activation function for the output layer is the sigmoid function. (1 mark)

The sigmoid function outputs a probability between 0 and 1 for each output node independently, allowing the model to predict the presence or absence of each anomaly separately. This is essential for multi-label problems where multiple outputs can be "on" at the same time. (1 mark)

**Question 4 [4 marks] (1 mark per curve/lr match + explanation)**

- **Curve 1** shows a clear divergence of the training, corresponding most likely to a high learning rate. A high value of learning leads to big gradient updates, which can lead to divergence. Because of this, the most likely learning rate is **0.5** (the highest value in the list).
- **Curve 3** shows a slow training, likely due to a small learning rate. A small learning rate leads to small gradient updates, which causes slow training. Because of this, the most likely learning rate is **0.001** (the smallest value in the list).
- **Curves 2 and 4** both show a fast convergence of the training to a specific loss value. However, curve 2 converges to a much higher value of loss. This is likely to be due to a higher learning rate than for curve 4, leading to overshooting the optimum. A training converging to a high loss value can be due to a learning rate higher than optimal, but not high enough to cause divergence. Therefore, we can say **curve 2** is likely linked to the learning rate of **0.1**, and **curve 4** to **0.01**.

Note that a small learning rate can also lead to getting stuck in a local optimum, therefore a higher loss. In that case, it may be also logical to think curve 2 can have a smaller learning rate than curve 4, respectfully 0.01 and 0.1. I gave partial marks when students explained it this way, and the explanation was consistent with other curves.

**Question 5 [4 marks]**

(a) **Masked attention in transformers**: (2 marks)

When training, the transformer typically tries to predict all the tokens in a known input sequence, based on the previous tokens in the sequence. The masking decoder's multi-head attention prevents each position from attending to future tokens, ensuring that predictions for a given position depend only on known outputs up to that point, preserving the autoregressive property during training.

(b) **Extra learnable embedding in ViT**: (2 marks)

The extra learnable embedding in ViT is used to summarise information about the patches in the sequence and fed to the MLP layer for classification. In other words, it is used to aggregate information from all patches for the final classification task.
The main advantage is efficiency as it allows the model to learn a global representation of the entire image during training, avoiding the need to process all patch embeddings individually at the output, through global pooling or attention weighted aggregation.

**Question 6 [3 marks]**

(a) Number of inputs in FC layer: ii. 180

(b) Let's calculate the shape of the feature map after each layer, using the following formulae.

Output size of convolution:

$$\left[\frac{n_H+2p-f}{s}+1\right]\times\left[\frac{n_W+2p-f}{s}+1\right]\times n'_C$$

Output size of pooling:

$$\left[\frac{n_H-f}{s}+1\right]\times\left[\frac{n_W-f}{s}+1\right]\times n_C$$

- Conv1:
  - Input: [35, 35, 3]
  - ((35 + 2*0 (valid padding) – 7) / 2) +1 = 14 + 1 = 15
  - Output: [15, 15, 10] (10 filters)
- MaxPool1:
  - Input: [15, 15, 10]
  - ((15 – 2) / 2) + 1 = 6.5 + 1 = 7.5 → 7
  - Output: [7, 7, 10]
- Conv2:
  - Input: [7, 7, 10]
  - Same spatial dimensions as using same padding.
  - Output: [7, 7, 20] (20 filters)
- MaxPool2:
  - Input: [7, 7, 20]
  - ((7 – 2) / 2) + 1 = 2.5 + 1 = 3.5 → 3
  - Output: [3, 3, 20]
- FC layer: 3 * 3 * 20 = 9 * 2 * 10 = 180